Faculty of Engineering and Technology

Computer Science Department

Database Systems 333

Project Report

Prepared by:

Malak Milhem **1220031**

Layal Hajji **1220871**

Instructor: Dr. Bassem Sayrafi

Section: 2

Date: Jan-2026

Table of Contents

Table of Contents II

Project Scope and Description 1

1.1. About Client 1

1.2. Entities (Tables) 2

1.3. Relationships 6

2. ER Diagram 7

2.1 Before Normalization 7

2.2 After Normalization 8

2.3 Entities Justification 9

3. Technology Description 10

4. Sample Queries 11

5. Normalization Analysis (1NF, 2NF, 3NF) 12

5.1 First Normal Form (1NF) 12

5.2 Second Normal Form (2NF) 12

5.3 Third Normal Form (3NF) 13

Project Scope and Description

The goal of this project is to design and implement a database management system for **AMIN Furniture Store** located in Ramallah, which specializes in selling home and office furniture. The system will handle all core business processes, including product availability tracking, supplier management, employee information, and maintaining organized records of customer orders.

The proposed database will serve as the core of an integrated management application, helping the client make informed business decisions through easy data access and query-based reporting.

1.1. About Client

- Client Name: AMIN Furniture Store
- Location: Ramallah, Palestine
- Business Type: Manufacture, import and installation of all types of home and office furniture.
- Operations: Selling various types of furniture such as sofas, tables, chairs, beds, and office equipment
- Client Needs:
  The store needs a reliable database system to manage customer orders, supplier deliveries, product stock, and staff information. The system should make it easier to find which items are in stock, which suppliers provide specific products, and track daily or monthly sales.

1.2. Entities (Tables)

The proposed system consists of ten core entities, each representing a real-world aspect of the business:

1. **Products**: stores information about each furniture item.

Main attributes:

- ProductID (Primary Key): unique ID for each product.
- ProductName
- Dimensions
- Color
- Material
- SellingPrice
- StockQuantity
- CategoryName.
- SupplierID (Foreign Key): identifies which supplier provides the product.
- DateAdded : date the product was added to the system.

2. **Customers** : manages client profiles and contact information.

Main attributes:

- CustomerID (Primary Key).
- FirstName
- LastName
- Email
- PhoneNumber
- Address.
- RegistrationDate : date the customer was registered.

3. **Employees**

Stores information about the employees who work at the store, including sales staff, managers, and delivery personnel.

Main attributes:

- EmployeeID (Primary Key): unique identifier for each employee.
- FirstName
- LastName
- Position
- HireDate.
- Salary : monthly salary of the employee.
- PhoneNumber
- Email.

4. **Orders**: records customer purchase transactions.

   Main attributes:

- OrderID (Primary Key).
- CustomerID (Foreign Key) : identifies who placed the order.
- EmployeeID (Foreign Key) : identifies who handled the order.
- OrderDate
- Status.
  Each order summarizes one transaction, while the specific products are stored in Order_ product.

5. **Order_Product** : links orders and products, tracking quantities and prices.

Main attributes:

- OrderDetailID (Primary Key).
- OrderID (Foreign Key) : links to the order.
- ProductID (Foreign Key) : identifies which product was purchased.
- Quantity
- PricePerUnit.

6. **Suppliers** : maintains supplier information and contacts.

Main attributes:

- SupplierID (Primary Key).
- SupplierName
- ContactPerson
- PhoneNumber
- Email.

7. **Purchases:** records what the store buys from suppliers, not just what customers buy like in **Orders** entity.

Main attributes:

- PurchaseID (Primary Key).
- SupplierID (Primary Key).

- PurchaseDate
- TotalCost
- ReceivedBy (Foreign Key) : links to EmployeeID.

8. **Purchase_Product:** A purchase contains multiple products.

Main attributes:

- PurchaseDetailID (Primary Key).
- PurchaseID (Foreign Key)
- ProductID (Foreign Key)
- QuantityPurchased
- CostPerUnit

9. **Payments:**

Main attributes:

- PaymentID (Primary Key).
- OrderID (Foreign Key)
- PaymentDate
- AmountPaid
- PaymentMethod : Cash, Card, E-payment, Bank Transfer.

10. **Delivery**

Main attributes:

- DeliveryID (Primary Key).
- OrderID (Foreign Key) – the order being delivered
- EmployeeID (Foreign Key) – delivery person responsible

- DeliveryAddress
- ScheduledDate
- DeliveryDate
- Status : Scheduled, Out for Delivery, Delivered, Failed

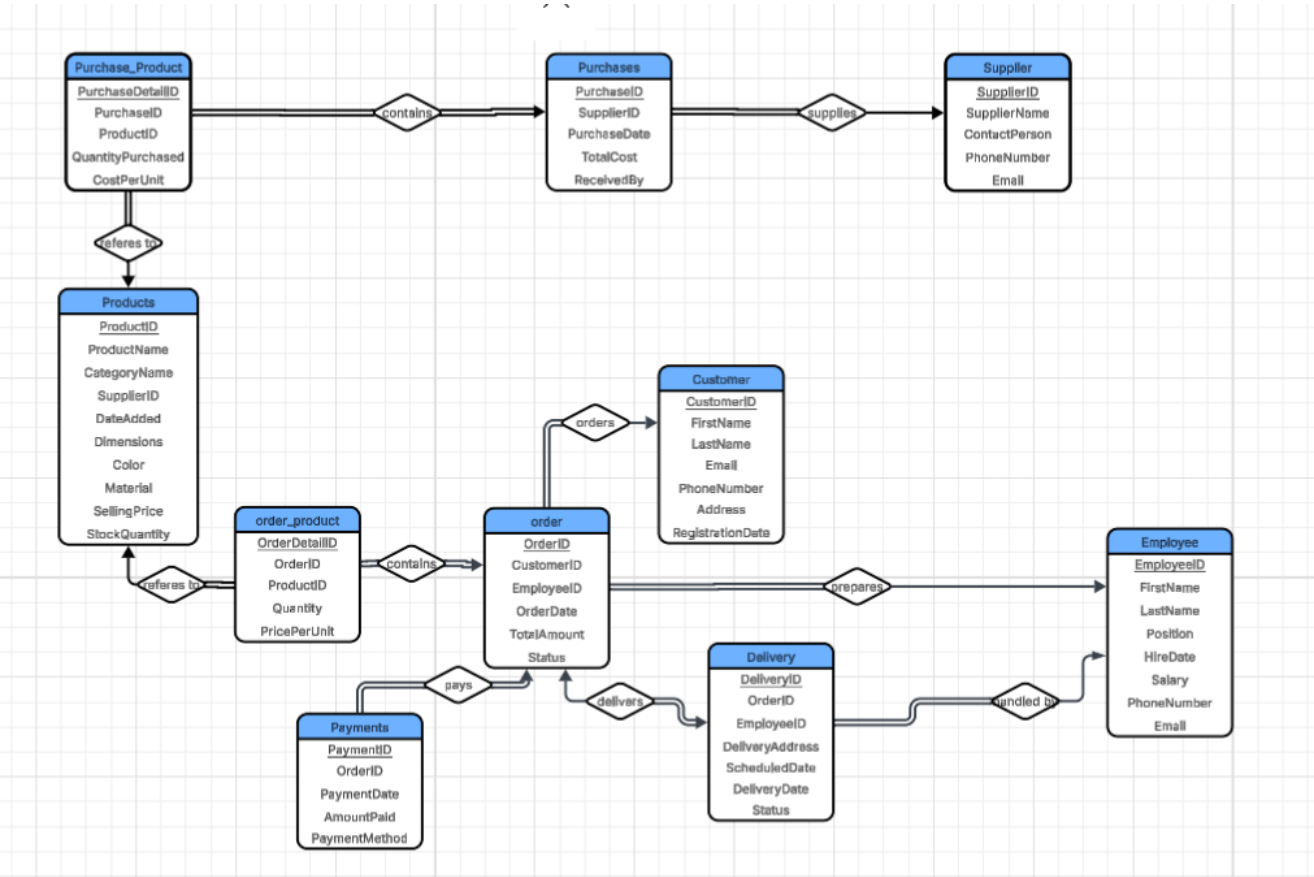
1.3. Relationships

- A **Supplier** can provide one or more **Products**, the **product** provided by one **supplier.**
- A **Customer** can place multiple **Orders**, but each order belongs to one customer.
- Each **order** is handled by one **Employee,** each employee can handle many orders.
- The **order** can have many products, and a **product** can be in many orders.
  It is a many to many relationship implemented via (**Order_product**)
- Each **Order** can have multiple **Order_product**, products recorded as individual order details.
- A **Products** can appear in many **Order_product** for different orders.
- One **Supplier** has many **Purchase** transactions.
- A **Purchase** contains many **Products**, A **Product** can appear in many **Purchase** batches.
  It is a many to many relationship implemented via (**Purchase_Product**)
- One **Employee** signs off receiving a **Purchase** shipment.
- An **Order** can have multiple **Payments**.
- An **Order** have one **Delivery,** Each **Delivery** belongs to exactly one **Order.**
- One **Employee** (delivery staff) can handle **many Deliveries**, Each **Delivery** is handled by **one Employee**

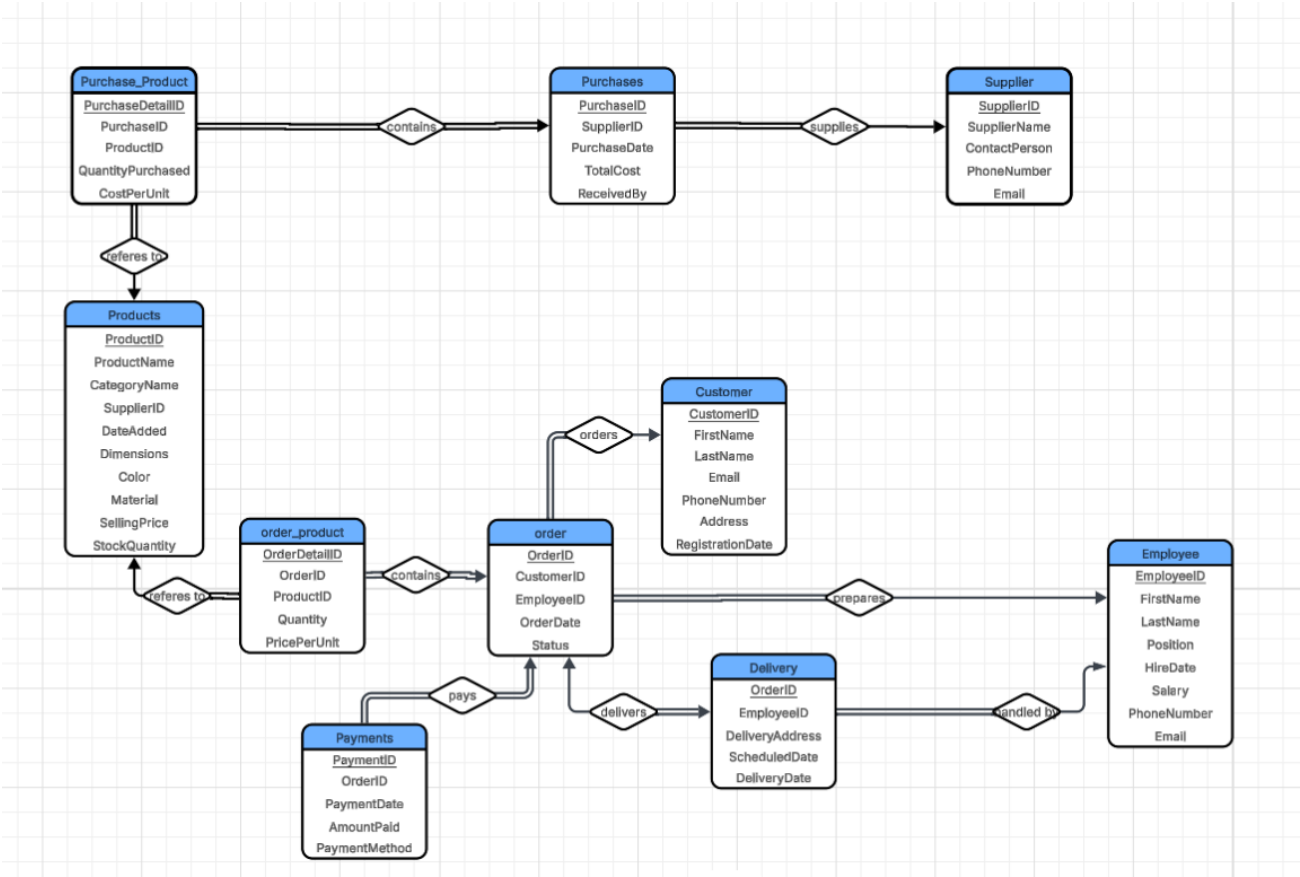
2. ER Diagram

The Entity-Relationship (ER) Diagram represents the conceptual structure of the database. It includes all entities, their attributes, and the relationships between them.

2.1 Before Normalization



2.2 After Normalization



2.3 Entities Justification

- **Products**

Products are the main items sold by the furniture store. The system must store a wide range of information (dimensions, material, supplier, stock levels) to support customer inquiries, inventory tracking, and pricing updates.

- **Customers**

Customer data enables the store to track purchase history, manage delivery information, and analyze customer buying behavior.

- **Employees**

Orders are processed by employees, so recording staff data is essential for accountability, performance tracking, and payroll integration.

- **Orders**

Orders are the core business transactions. The system records who placed the order, who handled it, and the financial amount.

- **Order_Product**

Orders and products have a many-to-many relationship. This table enables: Multiple products per order, multiple orders for the same product, and tracking product quantities.

- **Suppliers**

Supplier information is needed to track which products come from which supplier and to manage shipments and re-stocking.

- **Payment**

Payment entity records how a specific order was paid for. Separating *Payment* from *order* is essential because one order may require multiple payments, and different orders may use different payment methods. This entity ensures financial tracking, auditing, and secure record-keeping.

- **Purchase**

The *Purchase* entity represents the transactions where the store buys products from. It allows the system to track which supplier provided which products, and the total cost for the store. This entity is essential for managing inventory replenishment and keeping an accurate history of supplier transactions. Linking each purchase to an *Employee* through **ReceivedBy** ensures traceability of who verified or received the goods.

- **Delivery**

The *Delivery* entity records the process of delivering customer orders to their final destination. It stores essential information about delivery status, assigned delivery staff, delivery dates, and the address where the order is delivered.

3. Technology Description

The database will be developed using **MySQL**, which provides reliable data management and supports SQL for efficient querying and reporting. The frontend user interface (UI) will be designed using **HTML** for structure and **CSS** for styling. For backend functionality, the **Flask** framework in **Python** will handle server-side logic.

4. Sample Queries

- Retrieve the names, positions, and salaries of staff members, sorted by staff name.
- Retrieve all products with quantity less than 10 (low stock items).
- Retrieve the total quantity of products currently in stock.
- Retrieve the most sold product category.
- Retrieve the title, category, and price of all products supplied by a specific supplier.
- Retrieve the total number of products supplied by each supplier, sorted by supplier name.
- Retrieve a list of suppliers who delivered items within the last given days.
- Retrieve the total revenue generated by each employee.
- Retrieve all orders handled by a specific employee.
- Retrieve all products and their supplier names, sorted by category.
- Retrieve all products in the “Bedroom” category available in stock.
- Retrieve the total number of items sold per product.
- Retrieve all orders placed by a specific customer.
- Retrieve customer details along with their total number of purchases.
- Retrieve all customers who purchased products worth more than a given total.
- Retrieve the customer who spent the most money overall.
- Retrieve the top 5 best-selling products based on quantity sold.
- Retrieve the total sales (in NIS) generated for each product category.
- Retrieve employees with the number of their processed orders.
- Retrieve the details of the highest-priced product.
- Retrieve the details of the lowest-priced product.
- Retrieve the date and total amount of the most 5 recent orders.
- Retrieve all products sold between two given dates.

5. Normalization Analysis (1NF, 2NF, 3NF)

This section explains how the database design was evaluated against the normal forms and justifies the changes made during normalization.

5.1 First Normal Form (1NF)

A relation is in **First Normal Form (1NF)** if:

- All attributes contain **atomic (indivisible) values**
- There are **no multivalued or repeating attributes**

Check:

- All entities in the ERD use atomic attributes (e.g., Quantity, PricePerUnit, PhoneNumber).
- Many-to-many relationships (Orders–Products, Purchases–Products) are resolved using associative tables (Order_Product, Purchase_Product).
- No repeating groups or composite attributes exist.

Conclusion:

The database **satisfies 1NF** both before and after normalization.

5.2 Second Normal Form (2NF)

A relation is in **Second Normal Form (2NF)** if:

- It is already in 1NF
- There are **no partial dependencies**, meaning:
    - Non-key attributes must depend on the **entire primary key**, not part of it

Check:

- Associative tables (Order_Product, Purchase_Product) use surrogate primary keys (OrderDetailID, PurchaseDetailID).
- Attributes such as Quantity and PricePerUnit depend on the full key, not partially on OrderID or ProductID.
- No non-key attribute depends on only part of a composite key.

Conclusion:

The database **satisfies 2NF**.

5.3 Third Normal Form (3NF)

A relation is in **Third Normal Form (3NF)** if:

- It is already in 2NF
- There are **no transitive dependencies**, meaning:
    - Non-key attributes must depend **only on the primary key**, not on other non-key attributes

Identified Violations and Fixes:

During the **Third Normal Form (3NF)** analysis, several transitive and derived dependencies were identified and corrected. Initially, the TotalAmount attribute was stored in the Order table. However, this value is derived from the quantities and prices stored in Order_Product. Keeping it in Order created a transitive dependency and could lead to update and inconsistency anomalies. For this reason, TotalAmount was removed and is now calculated dynamically when needed.

Regarding the Status attribute, it was removed from the Delivery table and kept in the Order table. This change was made to avoid introducing a transitive dependency that would arise if the status were to be duplicated across multiple entities. The order status is directly tied to the order itself, so placing it in the Order table ensures that it follows the correct functional dependency rules for 3NF.

Finally, DeliveryID was removed from the Delivery table. Since there is a one-to-one relationship between Order and Delivery, the delivery can be uniquely identified by OrderID. Keeping a separate DeliveryID introduced an unnecessary dependency, violating 3NF. Using OrderID as the primary key for Delivery ensures that all non-key attributes depend directly on the primary key.

After applying these changes, all relations satisfy **Third Normal Form (3NF)**. Each non-key attribute depends only on the primary key, redundancy is eliminated, and update anomalies are prevented.